

Encapsulation and Abstraction

Example (1):

Create a **Lamp** class with private variables, and public methods to perform turn on and turn off and change color tasks. Ensure encapsulation and abstraction concepts founded in your program.

```
#include <iostream>
#include <string>
using namespace std;

class Lamp {
private:
    bool isOn;          // state of lamp is hiding
    string color;        // lamp color is hiding also

public:
    // constructor assignment is in the body of the function
    Lamp() {
        isOn=false;
        color="White";
    } // default color

    void turnOn() {
        isOn = true;
        cout << "Lamp is ON ??, Color: " << color << endl;
    }

    void turnOff() {
        isOn = false;
        cout << "Lamp is OFF ?? " << endl;
    }

    void setColor(string newColor) {
        color = newColor;
        cout << "Lamp color changed to: " << color << endl;
    }

    void showColor() {
        cout << "Current lamp color: " << color << endl;
    }
};

int main() {
    Lamp lamp;

    lamp.turnOn(); // turn on the lamp with default
    color
```

Output:
Lamp is ON ??, Color: White
Lamp color changed to: Red
Current lamp color: Red
Lamp is OFF ??

```
lamp.setColor("Red");           // change the color
lamp.showColor();               // display the current color
lamp.turnOff();                 // turn of the lamp

return 0;
}
```

Example 2:

In the following code, which line will cause a compilation error, and why?

```
#include <iostream>
#include <string>
using namespace std;
class Student {
private:
    int age;

public:
    Student(int a) {
        age = a;
    }

    int getAge() const {
        return age;
    }

    void setAge(int a) {
        age = a;
    }
};

int main() {
    const Student s(20);

    // Line 1
    cout << s.getAge();

    // Line 2
    s.setAge(25);

    return 0;
}
```